Task 2.1:

Part 1:

a) Webhooks are automatic event driven mechanisms. This means that when a sensor reading is taken the ESP32 triggers a HTTP POST/Get request to ThingSpeak API, passing data to as URL parameters. This makes them more efficient for real time event triggered Applications. Some examples a webhooks are things like ecommerce payments like PayPal to confirm that payment has been successful or GitHub triggering an automatic build pipeline when code is pushed.

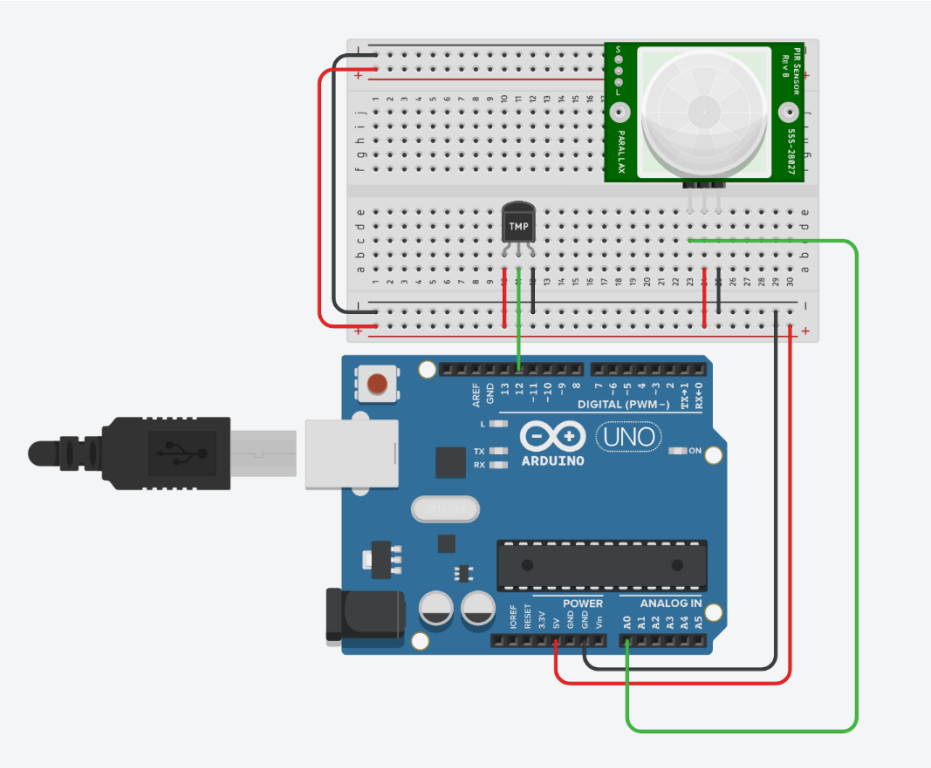
b) In this projects mechanism uses an esp32 microcontroller to connect the sensor to ThingSpeak over wifi. This is code to export data to ThingSpeak when an IoT device is connected to a Wi-Fi connection. It works by connecting to the internet and using an API key to access the end point API via a webhook HTTP request and gain permission to write to it.

```
WiFiClient client;
unsigned long myChannelNumber = 3286547; // Replace with your Channel ID
const char *myWriteAPIKey = "CK3HB4WORSVO66DS"; // Replace with your Write API Key

void setup() {
  Serial.begin(9600);
  pinMode(pirPin, INPUT);
  dht.begin();

  WiFi.begin("-----------", "-----------"); // Connect to Wi-Fi
  Serial.print("Connecting to WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");

  ThingSpeak.begin(client); // Initialize ThingSpeak
}
```

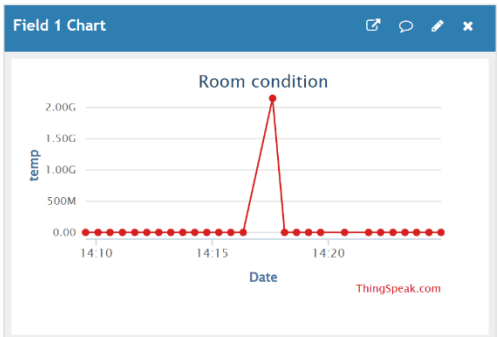
c) The ThingSpeak.writefields() acts a webhook trigger by constructing a HTTP GET request to api.thingspeak.com/update, appending the API key and sensor value as parameters. This causes the device to push data without any request from the server's side.

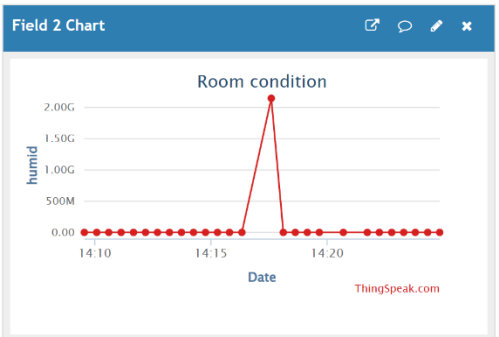
```
  ThingSpeak.setField(1, Temp);
  ThingSpeak.setField(2, Humid);
  ThingSpeak.setField(3, Pir);

  int result = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
  if (result == 200) {
    Serial.println("ThingSpeak update successful");
  } else {
    Serial.print("ThingSpeak update failed. Code: ");
    Serial.println(result);
  }

```

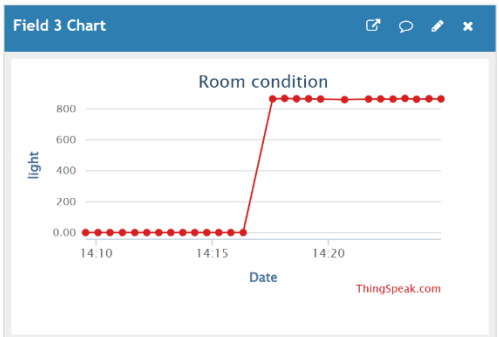
Part 2:



Part 3:







The graphs a displaying erroneous reading in places likely due to sensor instability during use. In production systems these reading would be filtered using threshold checks or moving average before uploading.

GitHub: https://github.com/charlieM044/SIT210